ESPECIFICACIÓN DEL DISEÑO DETALLADO
LOW-LEVEL DESIGN (LLD)

FitPaisa

Plataforma Web de Entrenamiento y Nutrición Personalizada

Proyecto	FitPaisa
Documento	Diseño Detallado del Software (DD/LLD)
Versión	1.0
Fecha	Marzo 2026
Autor	Javier Andrés García Vargas
Basado en	ERS + DAS + HLD (v1.0)
Estándar	ISO/IEC/IEEE 29148

1. Propósito

El presente documento constituye la Especificación del Diseño Detallado (DD/LLD) del sistema FitPaisa, una plataforma web integral para la gestión del entrenamiento físico y la nutrición personalizada. Su objetivo es traducir las decisiones arquitectónicas de alto nivel (HLD) y los requisitos funcionales (ERS) en especificaciones técnicas concretas y directamente implementables.

Este documento define de forma exhaustiva:

- Las clases, atributos, métodos y responsabilidades de cada componente de software.
- Los algoritmos y flujos internos de procesamiento de datos.
- El diseño lógico de la base de datos relacional.
- Las interfaces entre los módulos del sistema.
- Las estrategias de manejo de errores y consideraciones de seguridad.
- La trazabilidad completa desde los requisitos (ERS) hasta el diseño detallado.

Este documento sirve como guía directa para el equipo de desarrollo y como referencia para la validación del sistema.

2. Alcance

El diseño detallado abarca la totalidad de los subsistemas identificados en el Documento de Arquitectura del Software (DAS) y el Diseño de Alto Nivel (HLD), organizados en torno a los ocho módulos funcionales de FitPaisa:

Módulo	Descripción	RF Cubiertos
Gestión de Sesiones y Roles	Autenticación, autorización y control de acceso	RF1, RF10, RF12
Gestión de Perfiles	Datos físicos, objetivos y fotografías de progreso	RF2, RF8
Motor de Nutrición	Registro de comidas y cálculo de macronutrientes	RF3, RF4
Gestión de Entrenamientos	Rutinas, ejercicios, series, repeticiones y cargas	RF3, RF5, RF6
Motor de Estadísticas	Tendencias, gráficos y evolución física	RF8
Mensajería Interna	Comunicación usuario-entrenador	RF7
Pasarela de Pagos	Suscripciones Premium con Stripe/PayPal	RF11
Servicio de Notificaciones	Correos y alertas automáticas	RF9

3. Arquitectura Interna

FitPaisa implementa una arquitectura Cliente-Servidor con el patrón MVC (Modelo-Vista-Controlador), organizada en capas bien diferenciadas que garantizan la separación de responsabilidades, la mantenibilidad y la escalabilidad del sistema.

3.1 Estructura de Capas

Capa	Tecnología	Responsabilidad
Vista (Frontend)	HTML5, CSS3, JavaScript	Renderizado de UI, captura de eventos, peticiones REST
Controlador (Backend API)	Lógica de negocio web estándar	Gestión de peticiones, validación, coordinación de módulos
Modelo (Business Logic)	Capa de servicio interna	Reglas de negocio, cálculo de macros, lógica de planes
Repositorio (Data Access)	ORM / SQL	Abstracción del acceso a datos, consultas CRUD
Base de Datos	PostgreSQL / MySQL	Persistencia con integridad ACID

3.2 Flujo de Comunicación

El flujo estándar de una petición sigue la secuencia: Frontend → API REST (HTTPS/JSON) → Controlador → Servicio → Repositorio → Base de Datos, con la respuesta recorriendo el camino inverso. Las comunicaciones con servicios externos (Stripe, PayPal, SMTP) se realizan de forma asíncrona desde la capa de Servicio.

4. Diseño Detallado por Subsistema

4.1 Subsistema de Gestión de Sesiones y Roles

Este subsistema implementa los requisitos RF1, RF10 y RF12 de la ERS, garantizando el acceso seguro a la plataforma mediante autenticación basada en correo electrónico y contraseña, con sesiones gestionadas mediante tokens JWT.

4.1.1 Clase: User

Atributo	Tipo	Descripción	Restricción
userId	Integer	Identificador único del usuario	PK, Auto-increment
email	String(150)	Correo electrónico de acceso	Único, NOT NULL
passwordHash	String(255)	Contraseña cifrada (bcrypt)	NOT NULL
role	Enum	Rol del usuario: USER, COACH, ADMIN	NOT NULL, Default: USER
isActive	Boolean	Estado de la cuenta	Default: true
createdAt	DateTime	Fecha de registro	NOT NULL

Métodos de la clase User:

- login(): Valida credenciales y genera token de sesión JWT.
- logout(): Invalida el token activo y cierra la sesión.
- register(): Crea la cuenta, cifra la contraseña y valida el formato del correo.
- changePassword(oldPwd, newPwd): Verifica la contraseña actual y actualiza el hash.
- hasPermission(action): Verifica si el rol del usuario permite ejecutar la acción indicada.

4.1.2 Clase: AuthService

Responsabilidad: gestión centralizada de la autenticación y autorización.

Método	Parámetros	Retorno	Descripción
authenticate()	email, password	AuthResult	Verifica credenciales contra la BD

Método	Parámetros	Retorno	Descripción
generateToken()	user: User	String (JWT)	Crea token firmado con expiración
validateToken()	token: String	UserPayload | null	Verifica firma y vigencia del token
refreshToken()	token: String	String (JWT)	Renueva token antes de expiración
assignRole()	userId, role	Boolean	Cambia el rol de un usuario (solo Admin)

4.1.3 Algoritmo de Autenticación (RF1)

El proceso de login sigue los siguientes pasos en estricto orden:

1. Recibir email y contraseña desde el formulario del Frontend.
2. Buscar el usuario en la BD por email. Si no existe → retornar error 401.
3. Verificar que isActive = true. Si inactivo → retornar error 403.
4. Comparar contraseña recibida con passwordHash usando bcrypt.compare().
5. Si no coincide → incrementar contador de intentos fallidos. Si ≥ 5 → bloquear temporalmente.
6. Si coincide → generar token JWT con payload: {userId, email, role, exp: +2h}.
7. Registrar timestamp de último acceso y retornar token + datos de sesión.

4.2 Subsistema de Gestión de Perfiles

Este subsistema cubre los requisitos RF2 y RF8, permitiendo a los usuarios gestionar su información física y de progreso.

4.2.1 Clase: Profile

Atributo	Tipo	Descripción	Restricción
profileId	Integer	Identificador del perfil	PK, Auto-increment
userId	Integer (FK)	Referencia al usuario propietario	FK → Users, Único
weight	Decimal(5,2)	Peso en kilogramos	NOT NULL, > 0
height	Decimal(5,2)	Talla en centímetros	NOT NULL, > 0
age	Integer	Edad en años	NOT NULL, > 0
gender	Enum	MALE, FEMALE, OTHER	NOT NULL
objective	Enum	LOSE_WEIGHT, GAIN_MUSCLE, MAINTAIN, IMPROVE_HEALTH	NOT NULL

Atributo	Tipo	Descripción	Restricción
activityLevel	Enum	SEDENTARY, LIGHT, MODERATE, ACTIVE, VERY_ACTIVE	NOT NULL
updatedAt	DateTime	Última actualización	NOT NULL

4.2.2 Clase: ProgressPhoto

Atributo	Tipo	Descripción
photoId	Integer	Identificador de la fotografía (PK)
profileId	Integer (FK)	Referencia al perfil
fileUrl	String(500)	URL de almacenamiento de la imagen
uploadedAt	DateTime	Fecha y hora de subida
notes	Text	Notas opcionales del usuario

4.2.3 Clase: ProfileService

Métodos principales:

- createProfile(userId, data): Crea el perfil físico inicial del usuario.
- updateProfile(profileId, data): Actualiza datos físicos y objetivos.
- getProfileStats(profileId): Retorna el historial de peso y medidas.
- uploadProgressPhoto(profileId, file): Valida, almacena y registra la fotografía.
- calculateBMI(weight, height): Calcula el Índice de Masa Corporal.

4.3 Subsistema del Motor de Nutrición

Este subsistema implementa los requisitos RF3 y RF4, gestionando el registro diario de comidas y el cálculo automático de macronutrientes basado en el perfil del usuario.

4.3.1 Clase: FoodEntry

Atributo	Tipo	Descripción	Restricción
entryId	Integer	Identificador del registro de comida	PK, Auto-increment
profileId	Integer (FK)	Perfil del usuario	FK → Profiles, NOT NULL
foodName	String(200)	Nombre del alimento registrado	NOT NULL
portionGrams	Decimal(6,2)	Cantidad en gramos consumida	NOT NULL, > 0
calories	Decimal(7,2)	Calorías calculadas	NOT NULL

Atributo	Tipo	Descripción	Restricción
protein	Decimal(6,2)	Proteínas en gramos	NOT NULL
carbs	Decimal(6,2)	Carbohidratos en gramos	NOT NULL
fat	Decimal(6,2)	Grasas en gramos	NOT NULL
mealType	Enum	BREAKFAST, LUNCH, DINNER, SNACK	NOT NULL
logDate	Date	Fecha del registro	NOT NULL

4.3.2 Clase: NutritionService

Método	Parámetros	Retorno	Descripción
logFood()	profileId, foodData	FoodEntry	Registra una ingesta y calcula macros
getDailyLog()	profileId, date	FoodEntry[]	Obtiene todos los registros del día
getDailySummary()	profileId, date	MacroSummary	Suma total de macros del día
calculateMacros()	food, grams	MacroResult	Calcula macros proporcionalmente
getDailyTarget()	profile: Profile	MacroTarget	Calcula objetivos diarios según perfil
getWeeklyTrend()	profileId	TrendData[]	Retorna evolución calórica semanal

4.3.3 Algoritmo de Cálculo de Macronutrientes

El motor de macronutrientes calcula los objetivos diarios usando las fórmulas estándar de nutrición deportiva:

Paso 1 — Cálculo del Metabolismo Basal (TMB) con la ecuación de Mifflin-St Jeor:

- Hombre: TMB = (10 × peso_kg) + (6.25 × talla_cm) − (5 × edad) + 5
- Mujer: TMB = (10 × peso_kg) + (6.25 × talla_cm) − (5 × edad) − 161

Paso 2 — Gasto Energético Total (GET) = TMB × Factor de Actividad:

Nivel de Actividad	Factor
SEDENTARY	1.2
LIGHT	1.375
MODERATE	1.55
ACTIVE	1.725

Nivel de Actividad	Factor
VERY_ACTIVE	1.9

Paso 3 — Ajuste por objetivo: LOSE_WEIGHT → GET − 300 kcal | GAIN_MUSCLE → GET + 300 kcal | MAINTAIN → GET

Paso 4 — Distribución de macros: Proteínas 30% | Carbohidratos 45% | Grasas 25% (valores ajustables por entrenador).

4.4 Subsistema de Gestión de Entrenamientos

Implementa los requisitos RF3, RF5 y RF6, permitiendo el registro y gestión de rutinas de entrenamiento, tanto por parte del usuario como del entrenador asignado.

4.4.1 Clase: WorkoutPlan

Atributo	Tipo	Descripción	Restricción
planId	Integer	Identificador del plan	PK, Auto-increment
coachId	Integer (FK)	Entrenador creador del plan	FK → Users(role=COACH)
userId	Integer (FK)	Usuario asignado al plan	FK → Users
name	String(200)	Nombre descriptivo del plan	NOT NULL
status	Enum	DRAFT, PENDING_APPROVAL, ACTIVE, ARCHIVED	NOT NULL, Default: DRAFT
startDate	Date	Fecha de inicio del plan	NOT NULL
endDate	Date	Fecha de fin prevista	Nullable

4.4.2 Clase: Exercise

Atributo	Tipo	Descripción
exerciseId	Integer	Identificador del ejercicio (PK)
planId	Integer (FK)	Plan al que pertenece el ejercicio
name	String(200)	Nombre del ejercicio
sets	Integer	Número de series
reps	Integer	Repeticiones por serie
loadKg	Decimal(5,2)	Carga utilizada en kg (nullable para ejercicios de peso corporal)
restSeconds	Integer	Tiempo de descanso entre series en segundos

Atributo	Tipo	Descripción
dayOfWeek	Enum	MON, TUE, WED, THU, FRI, SAT, SUN
notes	Text	Instrucciones técnicas del ejercicio

4.4.3 Clase: WorkoutService

- createPlan(coachId, userId, planData): Crea un nuevo plan en estado DRAFT.
- submitForApproval(planId): Cambia estado a PENDING_APPROVAL y notifica al coach.
- approvePlan(planId, coachId): Valida permisos y activa el plan (estado ACTIVE).
- addExercise(planId, exerciseData): Agrega un ejercicio al plan.
- logWorkoutSession(userId, planId, sessionData): Registra una sesión completada con cargas reales.
- getPersonalRecord(userId, exerciseName): Retorna la carga máxima histórica de un ejercicio.
- getPlanHistory(userId): Retorna todos los planes del usuario ordenados por fecha.

4.5 Subsistema del Motor de Estadísticas

Este subsistema procesa los datos históricos de nutrición, entrenamientos y medidas físicas para generar gráficos y tendencias (RF8).

4.5.1 Clase: BodyLog

Atributo	Tipo	Descripción
logId	Integer	Identificador del registro (PK)
profileId	Integer (FK)	Perfil del usuario
weight	Decimal(5,2)	Peso registrado en kg
waist	Decimal(5,2)	Cintura en cm (nullable)
hips	Decimal(5,2)	Cadera en cm (nullable)
chest	Decimal(5,2)	Pecho en cm (nullable)
logDate	Date	Fecha del registro

4.5.2 Clase: StatsService

Método	Parámetros	Retorno	Descripción
getWeightTrend()	profileId, days	DataPoint[]	Evolución del peso en el período

Método	Parámetros	Retorno	Descripción
getMacroAverage()	profileId, days	MacroAvg	Promedio diario de macros consumidos
getWorkoutFrequency()	profileId, days	FreqData[]	Frecuencia de sesiones por semana
getProgressSummary()	profileId	ProgressSummary	Resumen integral del progreso
compareToTarget()	profileId, date	ComparisonResult	Compara ingesta real vs objetivo

4.6 Subsistema de Mensajería Interna

Implementa el requisito RF7, facilitando la comunicación directa y bidireccional entre usuarios y entrenadores dentro de la plataforma.

4.6.1 Clase: Message

Atributo	Tipo	Descripción	Restricción
messageId	Integer	Identificador del mensaje	PK, Auto-increment
senderId	Integer (FK)	Usuario remitente	FK → Users, NOT NULL
receiverId	Integer (FK)	Usuario destinatario	FK → Users, NOT NULL
content	Text	Contenido del mensaje	NOT NULL, Max 2000 chars
sentAt	DateTime	Fecha y hora de envío	NOT NULL
readAt	DateTime	Fecha y hora de lectura	Nullable
isDeleted	Boolean	Eliminación lógica	Default: false

4.6.2 Clase: MessagingService

- sendMessage(senderId, receiverId, content): Crea y persiste el mensaje. Valida que el par usuario-entrenador sea válido.
- getConversation(userId, otherUserId): Retorna el historial de mensajes ordenado por sentAt.
- markAsRead(messageId, userId): Actualiza readAt. Solo el destinatario puede marcar como leído.
- getUnreadCount(userId): Retorna el número de mensajes no leídos del usuario.

- deleteMessage(messageId, userId): Eliminación lógica (isDeleted = true). Solo el remitente puede eliminar.

4.7 Subsistema de Pasarela de Pagos

Implementa el requisito RF11, gestionando las suscripciones Premium mediante integración con Stripe y/o PayPal a través de sus APIs REST externas.

4.7.1 Clase: Subscription

Atributo	Tipo	Descripción	Restricción
subscriptionId	Integer	Identificador interno de la suscripción	PK, Auto-increment
userId	Integer (FK)	Usuario suscrito	FK → Users, NOT NULL
planType	Enum	FREE, PREMIUM_MONTHLY, PREMIUM_ANNUAL	NOT NULL
status	Enum	ACTIVE, CANCELLED, EXPIRED, PENDING	NOT NULL
externalRef	String(200)	ID de la transacción en Stripe/PayPal	Único, NOT NULL
startDate	Date	Fecha de inicio de la suscripción	NOT NULL
endDate	Date	Fecha de expiración	NOT NULL
amount	Decimal(8,2)	Importe cobrado en la divisa configurada	NOT NULL

4.7.2 Clase: PaymentService

Método	Parámetros	Retorno	Descripción
createCheckout()	userId, planType	CheckoutURL	Genera sesión de pago en Stripe/PayPal
handleWebhook()	event: PaymentEvent	void	Procesa callbacks de la pasarela externa
activateSubscription()	userId, externalRef	Subscription	Activa suscripción tras pago confirmado
cancelSubscription()	subscriptionId, userId	Boolean	Cancela y programa expiración al fin del período

Método	Parámetros	Retorno	Descripción
getActiveSubscription()	userId	Subscription | null	Retorna la suscripción vigente del usuario

4.7.3 Algoritmo de Procesamiento de Pago

8. El Frontend redirige al usuario a la URL de checkout generada por createCheckout().
9. El usuario completa el pago en la interfaz segura de Stripe/PayPal (PCI-DSS compliant).
10. La pasarela envía un webhook al endpoint /api/payments/webhook con el evento de pago.
11. handleWebhook() valida la firma del evento usando la clave secreta de la pasarela.
12. Si el evento es payment_intent.succeeded o PAYMENT.CAPTURE.COMPLETED → activateSubscription().
13. Si el evento indica fallo → persistir estado CANCELLED y notificar al usuario por email.
14. En ambos casos → registrar el evento completo en el log de auditoría.

4.8 Subsistema de Servicio de Notificaciones

Implementa el requisito RF9, enviando correos electrónicos automáticos para recordatorios de entrenamientos, alertas de nutrición y confirmaciones de transacciones.

4.8.1 Clase: Notification

Atributo	Tipo	Descripción
notificationId	Integer	Identificador de la notificación (PK)
userId	Integer (FK)	Destinatario de la notificación
type	Enum	WORKOUT_REMINDER, NUTRITION_ALERT, PAYMENT_CONFIRM, PLAN_UPDATE, MESSAGE_RECEIVED
channel	Enum	EMAIL, IN_APP
status	Enum	PENDING, SENT, FAILED
sentAt	DateTime	Fecha y hora de envío efectivo (nullable)
payload	JSON	Datos dinámicos para la plantilla del mensaje

4.8.2 Clase: NotificationService

- scheduleReminder(userId, type, scheduledAt): Programa el envío de un recordatorio.
- sendImmediate(userId, type, payload): Envía una notificación de forma inmediata.
- sendEmail(to, templateId, data): Llama al servicio SMTP externo con la plantilla correspondiente.
- processQueue(): Proceso background que procesa las notificaciones pendientes.
- markDelivered(notificationId): Actualiza el estado a SENT con el sentAt actual.

5. Diseño de Base de Datos (Nivel Lógico)

El modelo de datos relacional de FitPaisa implementa las entidades definidas en el HLD, con claves foráneas, restricciones de integridad y diseño normalizado (3FN).

5.1 Tablas Principales

USERS — Tabla central del sistema:

Campo	Tipo	Restricciones
user_id	INT	PK, AUTO_INCREMENT
email	VARCHAR(150)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
role	ENUM('USER','COACH','ADMIN')	NOT NULL, DEFAULT 'USER'
is_active	BOOLEAN	DEFAULT TRUE
created_at	DATETIME	NOT NULL, DEFAULT NOW()
last_login	DATETIME	NULLABLE

PROFILES — Datos físicos del usuario:

Campo	Tipo	Restricciones
profile_id	INT	PK, AUTO_INCREMENT
user_id	INT	FK → users(user_id), UNIQUE
weight	DECIMAL(5,2)	NOT NULL

Campo	Tipo	Restricciones
height	DECIMAL(5,2)	NOT NULL
age	INT	NOT NULL
gender	ENUM('MALE','FEMALE','OTHER')	NOT NULL
objective	ENUM('LOSE_WEIGHT','GAIN_MUSCLE','MAINTAIN','IMPROVE_HEALTH')	NOT NULL
activity_level	ENUM('SEDENTARY','LIGHT','MODERATE','ACTIVE','VERY_ACTIVE')	NOT NULL
updated_at	DATETIME	NOT NULL

FOOD_ENTRIES — Registro diario de nutrición:

Campo	Tipo	Restricciones
entry_id	INT	PK, AUTO_INCREMENT
profile_id	INT	FK → profiles(profile_id)
food_name	VARCHAR(200)	NOT NULL
portion_grams	DECIMAL(6,2)	NOT NULL
calories	DECIMAL(7,2)	NOT NULL
protein	DECIMAL(6,2)	NOT NULL
carbs	DECIMAL(6,2)	NOT NULL
fat	DECIMAL(6,2)	NOT NULL
meal_type	ENUM('BREAKFAST','LUNCH','DINNER','SNACK')	NOT NULL
log_date	DATE	NOT NULL

WORKOUT_PLANS — Planes de entrenamiento:

Campo	Tipo	Restricciones
plan_id	INT	PK, AUTO_INCREMENT
coach_id	INT	FK → users(user_id)

Campo	Tipo	Restricciones
user_id	INT	FK → users(user_id)
name	VARCHAR(200)	NOT NULL
status	ENUM('DRAFT','PENDING_APPROVAL','ACTIVE','ARCHIVED')	NOT NULL
start_date	DATE	NOT NULL
end_date	DATE	NULLABLE

EXERCISES — Ejercicios de un plan:

Campo	Tipo	Restricciones
exercise_id	INT	PK, AUTO_INCREMENT
plan_id	INT	FK → workout_plans(plan_id)
name	VARCHAR(200)	NOT NULL
sets	INT	NOT NULL
reps	INT	NOT NULL
load_kg	DECIMAL(5,2)	NULLABLE
rest_seconds	INT	NOT NULL
day_of_week	ENUM('MON','TUE','WED','THU','FRI','SAT','SUN')	NOT NULL

MESSAGES — Mensajería interna:

Campo	Tipo	Restricciones
message_id	INT	PK, AUTO_INCREMENT
sender_id	INT	FK → users(user_id)
receiver_id	INT	FK → users(user_id)
content	TEXT	NOT NULL, MAX 2000 chars
sent_at	DATETIME	NOT NULL, DEFAULT NOW()
read_at	DATETIME	NULLABLE

Campo	Tipo	Restricciones
is_deleted	BOOLEAN	DEFAULT FALSE

SUBSCRIPTIONS — Suscripciones de pago:

Campo	Tipo	Restricciones
subscription_id	INT	PK, AUTO_INCREMENT
user_id	INT	FK → users(user_id)
plan_type	ENUM('FREE','PREMIUM_MONTHLY','PREMIUM_ANNUAL')	NOT NULL
status	ENUM('ACTIVE','CANCELLED','EXPIRED','PENDING')	NOT NULL
external_ref	VARCHAR(200)	UNIQUE, NOT NULL
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL
amount	DECIMAL(8,2)	NOT NULL

5.2 Diagrama de Relaciones Simplificado

Las relaciones principales del modelo son: Users 1:1 Profiles | Profiles 1:N FoodEntries | Profiles 1:N BodyLogs | Users 1:N WorkoutPlans (como COACH) | Users 1:N WorkoutPlans (como USER) | WorkoutPlans 1:N Exercises | Users N:M Messages | Users 1:N Subscriptions | Profiles 1:N ProgressPhotos.

6. Interfaces Internas

Origen	Destino	Tipo	Protocolo / Mecanismo
Frontend (View)	Backend API (Controller)	Síncrona	HTTPS / REST / JSON
AuthService	UsersRepository	Síncrona	ORM (método interno)
NutritionService	FoodEntriesRepository	Síncrona	ORM (método interno)
WorkoutService	PlansRepository	Síncrona	ORM (método interno)

Origen	Destino	Tipo	Protocolo / Mecanismo
StatsService	Múltiples Repositories	Síncrona	Consultas SQL agregadas
PaymentService	Stripe / PayPal API	Síncrona (checkout) / Asíncrona (webhook)	REST API HTTPS
NotificationService	Servidor SMTP	Asíncrona (cola)	SMTP / Email API
MessagingService	Frontend (polling)	Síncrona periódica	REST / JSON

7. Manejo de Errores y Excepciones

7.1 Estrategia General

Todos los módulos implementan el patrón de manejo centralizado de errores. Las excepciones no controladas se capturan en el middleware global del controlador, que registra el error en el sistema de logs y retorna una respuesta estructurada al cliente con el código HTTP apropiado.

7.2 Catálogo de Errores por Módulo

Módulo	Escenario de Error	Código HTTP	Acción del Sistema
AuthService	Credenciales incorrectas (< 5 intentos)	401 Unauthorized	Respuesta genérica, no revelar motivo específico
AuthService	Cuenta bloqueada por intentos excesivos	403 Forbidden	Mensaje de bloqueo temporal + tiempo restante
AuthService	Token expirado o inválido	401 Unauthorized	Redirección al login desde el Frontend
ProfileService	Datos físicos fuera de rango	400 Bad Request	Lista de campos inválidos devuelta al cliente
NutritionService	Alimento no encontrado en la BD	404 Not Found	Sugerencia de registro manual del alimento
WorkoutService	Plan en estado no editable	409 Conflict	Mensaje indicando el estado actual del plan

Módulo	Escenario de Error	Código HTTP	Acción del Sistema
PaymentService	Pago rechazado por la pasarela	402 Payment Required	Notificación al usuario + log de auditoría
PaymentService	Webhook con firma inválida	400 Bad Request	Ignorar evento + alerta al sistema de monitoreo
MessagingService	Intento de mensaje entre no vinculados	403 Forbidden	Validación de relación usuario-entrenador

7.3 Política de Logs

- Nivel ERROR: Excepciones no controladas, fallos de pasarela de pago, errores de base de datos.
- Nivel WARN: Intentos de autenticación fallidos, accesos denegados por permisos.
- Nivel INFO: Transacciones exitosas, cambios de estado de pedidos, suscripciones activadas.
- Nivel DEBUG: Solo activo en entorno de desarrollo. Incluye payloads completos de peticiones.

8. Consideraciones de Seguridad

Área	Implementación	Referencia ERS
Contraseñas	Hash con bcrypt (factor de coste ≥ 12). Nunca almacenar en texto plano.	RF1, RNF (Seguridad)
Tokens de sesión	JWT firmados con clave secreta (HS256). Expiración de 2 horas.	RF1
Transmisión de datos	TLS 1.2 o superior en todas las comunicaciones cliente-servidor.	RNF (Seguridad)
Control de acceso	Middleware de autorización valida rol en cada endpoint protegido.	RF10
Validación de entradas	Sanitización y validación server-side en todos los controladores.	RNF (Seguridad)
Datos de pago	Nunca se almacenan datos de tarjeta. Delegado completamente a Stripe/PayPal (PCI-DSS).	RF11
Protección CSRF	Tokens CSRF en formularios de mutación de datos.	RNF (Seguridad)

Área	Implementación	Referencia ERS
Rate limiting	Límite de 100 peticiones/min por IP. Bloqueo tras 5 intentos de login fallidos.	RF1

9. Trazabilidad ERS → DAS → HLD → Diseño Detallado

RF (ERS)	Módulo HLD	Componente DAS	Clase/Servicio DD
RF1 – Registro y autenticación	Gestión de Sesiones y Roles	MVC - Controlador	User, AuthService
RF2 – Perfil físico del usuario	Gestión de Perfiles	MVC - Modelo	Profile, ProfileService
RF3 – Consultar plan personalizado	Gestión de Entrenamientos / Nutrición	MVC - Vista+Modelo	WorkoutPlan, FoodEntry
RF4 – Registro de comidas y macros	Motor de Nutrición	MVC - Modelo/Controlador	FoodEntry, NutritionService
RF5 – Registro de entrenamientos	Gestión de Entrenamientos	MVC - Modelo/Controlador	Exercise, WorkoutService
RF6 – Gestión de planes por entrenador	Gestión de Entrenamientos	MVC - Controlador	WorkoutPlan, WorkoutService
RF7 – Mensajería usuario-entrenador	Mensajería Interna	MVC - Controlador	Message, MessagingService
RF8 – Fotos y estadísticas de evolución	Motor de Estadísticas / Perfiles	MVC - Modelo	BodyLog, StatsService, ProgressPhoto
RF9 – Notificaciones y recordatorios	Servicio de Notificaciones	MVC - Controlador (Externo)	Notification, NotificationService
RF10 – Gestión de usuarios por Admin	Gestión de Sesiones y Roles	MVC - Controlador	AuthService (assignRole)
RF11 – Suscripciones de pago	Pasarela de Pagos	MVC - Controlador (Externo)	Subscription, PaymentService

RF (ERS)	Módulo HLD	Componente DAS	Clase/Servicio DD
RF12 – Registro en máx. 3 pasos	Interfaz de Usuario (Vista)	MVC - Vista	Frontend UX (flujo controlado)

10. Glosario Técnico

Término	Definición
JWT	JSON Web Token. Estándar abierto (RFC 7519) para transmitir información de forma segura entre partes como objeto JSON firmado.
bcrypt	Función criptográfica de hash adaptativa para contraseñas, diseñada para ser computacionalmente costosa y resistente a ataques de fuerza bruta.
ORM	Object-Relational Mapping. Técnica de programación que convierte datos entre sistemas de tipos incompatibles (objetos y tablas relacionales).
REST	Representational State Transfer. Estilo arquitectónico para el diseño de servicios web basado en HTTP y recursos identificados por URLs.

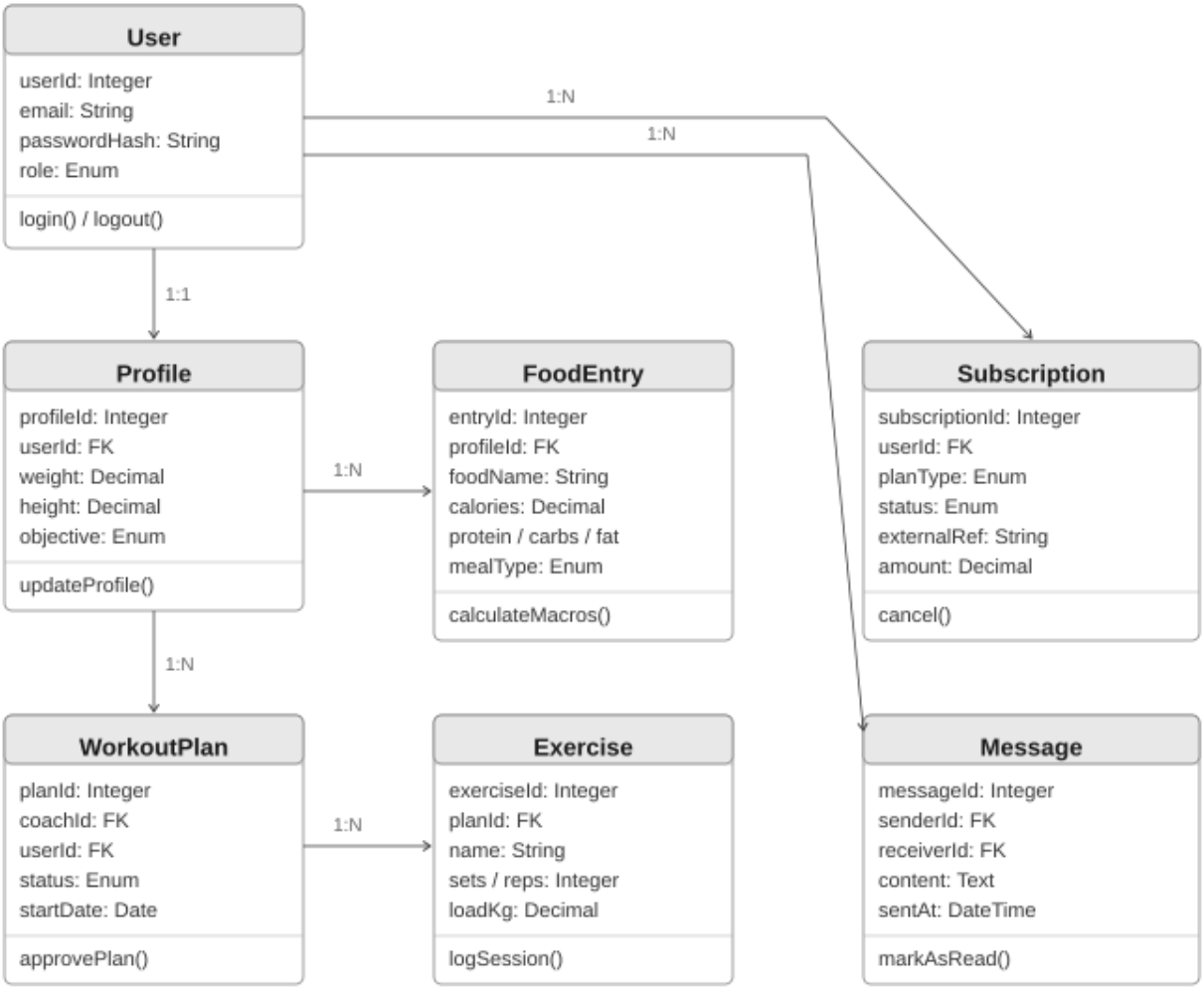
Término	Definición
MVC	Modelo-Vista-Controlador. Patrón de diseño que separa la aplicación en tres componentes: datos (Modelo), presentación (Vista) y lógica de control (Controlador).
TLS	Transport Layer Security. Protocolo criptográfico que proporciona comunicaciones seguras sobre redes informáticas (versión 1.2 o superior requerida).
Webhook	Mecanismo de comunicación en el que un servicio externo (como Stripe) envía una notificación HTTP a un endpoint predefinido cuando ocurre un evento.
TMB	Tasa de Metabolismo Basal. Energía mínima que el cuerpo necesita en reposo absoluto para mantener sus funciones vitales.
GET	Gasto Energético Total. Cálculo de las calorías totales que una persona necesita considerando su nivel de actividad física diaria.
PK / FK	Primary Key / Foreign Key. Claves primaria y foránea en el modelo relacional para identificar registros y establecer relaciones entre tablas.

11. Diagramas UML

Los siguientes diagramas documentan la arquitectura, estructura de clases, flujos de interacción y ciclos de vida del sistema FitPaisa.

11.1 Diagrama de Clases

Representa las clases principales del sistema, sus atributos, métodos y relaciones de asociación entre entidades.

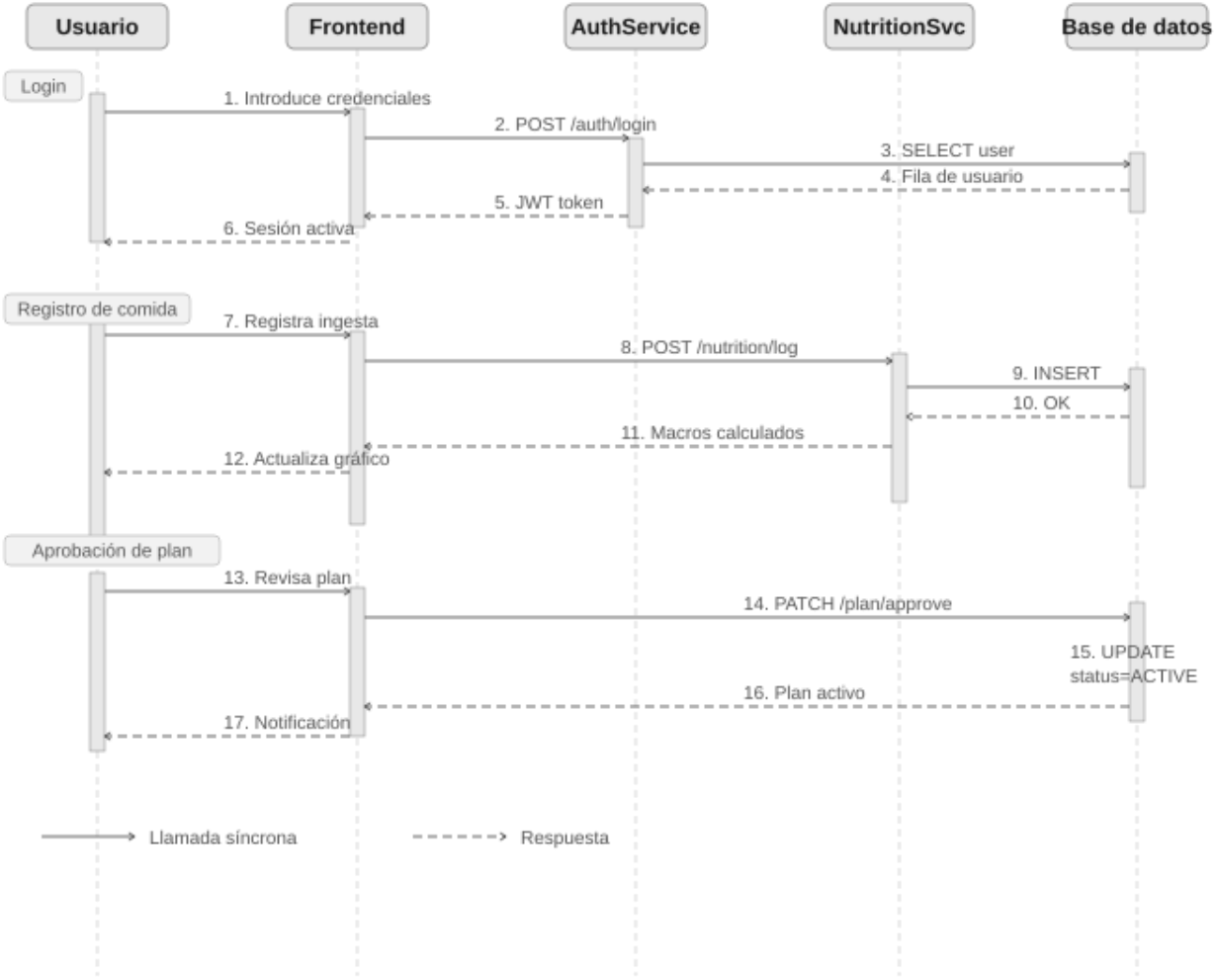


11.2 Diagrama de Componentes

Muestra la arquitectura en capas del sistema (Frontend, Backend API, Lógica de Negocio, Repositorio/BD) y sus dependencias con servicios externos.

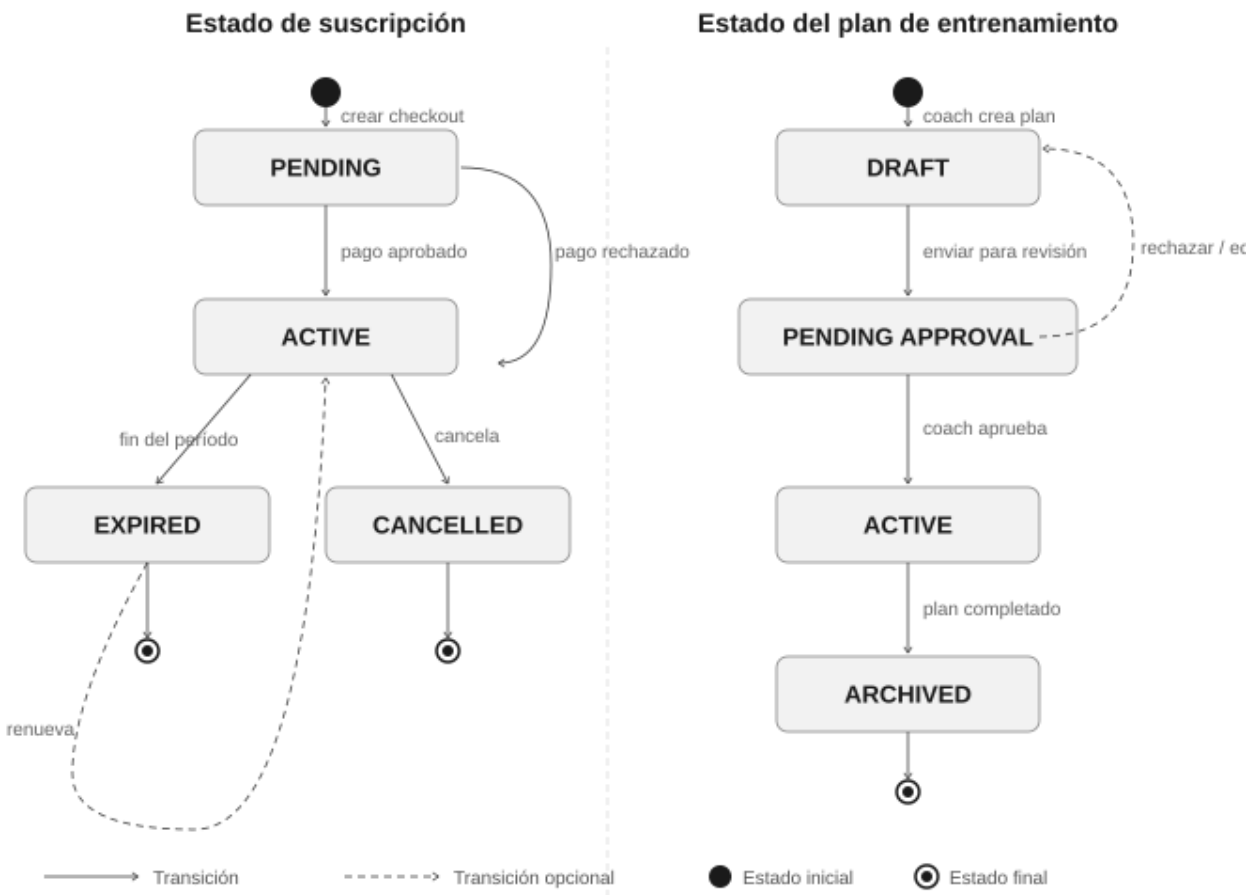
11.3 Diagrama de Secuencia

Ilustra tres flujos principales: autenticación de usuario (login), registro de ingesta con cálculo de macros, y aprobación de plan de entrenamiento por el entrenador.



11.4 Diagrama de Estados

Describe el ciclo de vida de dos entidades clave: la Suscripción (PENDING → ACTIVE → EXPIRED/CANCELLED) y el Plan de Entrenamiento (DRAFT → PENDING APPROVAL → ACTIVE → ARCHIVED).


Estado de suscripción
crear checkout
PENDING
pago aprobado
pago rechazado
ACTIVE
fin del periodo
cancela
EXPIRED
CANCELLED
renueva
Estado del plan de entrenamiento
coach crea plan
DRAFT
enviar para revisión
rechazar / ec
PENDING APPROVAL
coach aprueba
ACTIVE
plan completado
ARCHIVED
Transición
Transición opcional
Estado inicial
Estado final